

TASK TUNNEL

Android Intentional-Use App

Product differentiation, technical architecture, Play Store constraints,
and solo-developer implementation plan

Bottom line

A useful, differentiated Android version is feasible for one solo developer if the first release supports only a small number of apps and treats "compartmentalization" as feature detection + interception. A generic system that reliably rewrites arbitrary native apps is not feasible. The main long-term burden is maintaining per-app detectors as Instagram, YouTube, and Android change.

Research and feasibility review current to 20 September 2026

Product working name only - not a trademark or final brand recommendation

Executive Summary

The opportunity is not to build another timer or hard app blocker. The strongest product position is to protect a user's declared intention while keeping the useful parts of distracting apps available. This is supported by both user complaints and research: people dislike all-or-nothing blocking, feature-level interventions can reduce passive consumption while preserving useful features, and small moments of friction can materially reduce automatic app opening. [7][8][9][18]

Decision	Assessment	What it means
Can one solo developer build it?	YES - with constrained scope	Kotlin/Compose + AccessibilityService is enough for an MVP. No server or ML is required.
Can it truly modify Instagram/YouTube?	NO	A third-party app cannot remove native tabs. It can detect forbidden surfaces and overlay/exit them.
Can Task Tunnel feel effective anyway?	YES	If detection is fast and reliable, the user experiences a protected session even though the underlying app is unchanged.
Biggest engineering risk	Detector brittleness	Resource IDs, labels and UI structures can change after third-party app updates.
Biggest distribution risk	Play review + permissions	AccessibilityService requires a declaration, prominent disclosure, consent and policy-compliant behavior. [3]
Best MVP	2 apps, 2-4 tasks each	Start with Instagram + YouTube, English UI, local-only data, user-controlled override.

Recommended product thesis

"Your phone is useful. The rabbit holes are not." The app should distinguish intentional tasks from passive consumption rather than treating all screen time as bad.

Contents

- 1. Product problem and market gap
- 2. Competitive reality: what is already commoditized
- 3. Proposed product: the complete feature set
- 4. Solo-developer feasibility by feature
- 5. How native Task Tunnel works on Android
- 6. Recommended technical architecture
- 7. Reliability and maintenance strategy
- 8. Google Play and privacy constraints
- 9. MVP scope and build sequence
- 10. Testing, validation and success metrics
- 11. Monetization and positioning
- 12. Risks, kill criteria and final recommendation
- References

1. Product Problem and Market Gap

The category already contains excellent blockers. The remaining opportunity comes from frustrations that current tools only partially solve. Community evidence is anecdotal, so it should guide product discovery rather than be treated as causal proof. The recurring themes are still consistent enough to shape an MVP. [18][19][20]

User frustration	Why current approaches struggle	Product implication
"I need the useful part of the app."	Instagram combines DMs with Reels/Explore; YouTube combines tutorials with Shorts/recommendations.	Protect features and tasks, not entire apps.
"I learn to ignore the blocker."	Repeated prompts/countdowns become part of the habit loop; ScreenZen already counters this with increasing delays. [11]	Use varied, context-relevant friction; do not rely on one countdown forever.
"If it is strict, I uninstall it; if it is weak, I bypass it."	Hard blocking creates frustration; gentle blockers can become invisible.	Preserve an explicit escape hatch and focus on intentional choice rather than coercion.
"I deleted Instagram and moved to YouTube."	App-centric restrictions do not address cross-app stimulation seeking. [19]	Detect short cross-app switching bursts as a single distraction episode.
"I do not trust an app that can see my screen."	Accessibility access is powerful and understandably sensitive.	Local-first processing, minimal permissions, plain-language disclosure and visible service status.
"Why is a blocker a \$100/year subscription?"	Users often resist high recurring prices for simple blocking utilities. [20]	Keep core value inexpensive; reserve premium for higher-value insights or advanced controls.

Evidence that the direction can work

- The one sec field experiment found that a short opening intervention caused users to dismiss 36% of attempted target-app openings; opening attempts also fell over six weeks. [8]
- A Danish six-week field experiment with 269 teenagers found that reflection, planning or a six-second wait reduced social-media activity by roughly 31-36% without lowering reported satisfaction. [9]
- FinerMe found feature-level interventions could reduce passive consumption while preserving other uses of the same app, supporting the basic premise behind compartmentalization. [7]
- Time2Stop shows adaptive intervention timing can improve intervention accuracy and receptivity, but its effect on visit frequency was relatively modest. This argues against making ML the MVP centerpiece. [10]

2. Competitive Reality: What Is Already Commoditized

Product differentiation must come from the workflow and combination of features, not from claiming that individual mechanisms are new. The following capabilities already exist in notable products or open-source projects.

Capability	Already present	Implication for this product
Pause/delay before opening	one sec; ScreenZen; many blockers [8] [11]	Baseline feature, not differentiation.
Increasing delay with repeated opens	ScreenZen [11]	Do not market escalating friction as novel.
Random intervention types	one sec Android/iOS customization [12]	Variation alone is not novel.
Ask for intention	one sec on iOS [12]	Android execution can still be useful, but the concept exists.

Capability	Already present	Implication for this product
Ask how long / re-intervene	one sec "Always Ask" [13]	Session-window selection is baseline functionality.
Feature-level blocking	UNDOOMED; open-source blockers; FinerMe [7][14][16]	Feasible, but not novel by itself.
Messages-only / remove feeds	UNDOOMED [14]	Static communication-only modes already exist.
Strict anti-bypass modes	AppBlock [15]	Not a good primary differentiator.
Adaptive friction	Detoxly; Time2Stop research [10][16]	Avoid generic "AI adaptive blocker" positioning.

Where differentiation still looks meaningful

The strongest combination is: (1) user declares a task, (2) the allowed in-app surfaces change for that temporary session, (3) cross-app drift is treated as one distraction episode, and (4) the app explains the episode afterward as an attention timeline. I did not find a mainstream Android product clearly owning that whole workflow.

3. Proposed Product: Complete Feature Set

3.1 Purpose Gate

When a supported distracting app opens, the service presents a brief gate asking what the user came to do. The choices should be task-shaped, not moral judgments.

- Instagram examples: Reply/message, Post/share, Find a specific account/post, Browse intentionally.
- YouTube examples: Search/watch something specific, Subscriptions, Upload/manage, Browse intentionally.
- Optional session window: 5, 10, 20 minutes or custom. Intentional leisure is a valid choice; the product should not shame it.

3.2 Task Tunnel

The declared task creates a temporary policy for that app. The native app remains untouched; the accessibility layer watches the active surface and intervenes only when the user enters a surface outside the policy.

Example task	Allowed	Intercepted
Instagram - Reply	DM inbox, conversations	Reels, Explore, optionally Home feed
Instagram - Post	Composer and required navigation	Reels/Explore detours
YouTube - Find/watch	Search, long-form player, optionally Subscriptions	Shorts; optionally Home recommendations
YouTube - Browse intentionally	All selected surfaces within chosen window	Only session-expiry re-intervention

3.3 Surface Interception

A blocked surface should trigger an immediate accessibility overlay or a user-visible return action. The safest UX is transparent: "You opened Instagram to reply. Reels is outside this Task Tunnel." Give the user Return, End Tunnel, and Allow Anyway. Avoid secretly fighting the user or making the device feel broken.

3.4 Session Window and Re-intervention

The user may set an intended window. At expiry, re-intervene rather than forcibly terminating the app. Existing evidence suggests that a pause plus a meaningful choice can reduce automatic consumption. [8][9]

3.5 Drift Detection

Treat rapid movement among selected distraction apps as one behavioral episode. Example: Instagram -> Reddit -> YouTube within 60 seconds after abandoning or being intercepted in the first app. A lightweight intervention can say: "You have switched between 3 distraction apps in 54 seconds. Looking for something specific?"

This can be implemented with a rolling local event window. It does not require AI, content reading, or a server.

3.6 Anti-habituation Friction

Do not rely on one fixed countdown. Rotate a small library of interventions and prefer those that still cause useful behavior changes. one sec already randomizes intervention types, ScreenZen escalates delay, and Detoxly markets adaptive friction, so this is supporting functionality rather than the unique selling point. [11][12][16]

- MVP rule: deterministic rotation or simple scoring based on recent exposures and "continue vs leave" behavior.
- Later: personalize intervention selection, but keep it explainable and user-controlled.

3.7 Attention Debugger

Turn event logs into a narrative rather than a guilt-inducing screen-time chart. Example: "7:22 opened Instagram to message; 7:27 entered Reels; 7:50 switched to Reddit; 8:04 opened YouTube; episode ended 8:31." This makes cross-app drift visible and gives the product a memorable "debug your attention" identity.

3.8 Progress Without Purity

Avoid making a perfect streak the main success measure. Prefer rolling metrics such as intentional-session rate, distraction episodes per week, overrides, abandoned automatic opens, and trend direction. A bad day should not reset the user to zero.

3.9 Reliability Health Check

Always show whether protection is active. Accessibility-based products can fail when permissions are disabled, OEM battery management intervenes, or an app redesign breaks detection. one sec itself documents Android manufacturer compatibility and permission/background issues. [12]

- Status: Accessibility service on/off, Usage access if used, last detector heartbeat, supported app versions, detector confidence.
- Provide a one-tap diagnostic page and a non-sensitive "copy diagnostics" report for bug reports.

3.10 Local-first Privacy

Keep session events, tasks, surface detections and insights on-device by default. A digital-wellbeing app that inspects accessibility trees has a much higher trust burden than a normal utility. The first version does not need an account or cloud backend.

4. Solo-Developer Feasibility by Feature

Feature	Build difficulty	Maintenance	Play/policy risk	Solo-dev verdict
Purpose gate + task selection	Low	Low	Low	GREEN - straightforward Compose UI/state.

Feature	Build difficulty	Maintenance	Play/policy risk	Solo-dev verdict
Session window / re-intervention	Low	Low	Low-Med	GREEN - proven pattern.
Drift Detection	Low-Med	Low	Low	GREEN - rolling app-switch event logic.
Attention Debugger timeline	Low-Med	Low	Low	GREEN - local event aggregation.
Rule-based intervention variation	Low	Low	Low	GREEN - no ML needed.
YouTube Shorts surface detector	Medium	High	Medium	YELLOW - feasible but UI updates will break signatures.
Instagram Reels/Explore detector	Medium-High	High	Medium	YELLOW - same issue, more surface variation.
Task-specific native compartmentalization	Medium-High	High	Medium	YELLOW - feasible for a few explicit tasks/apps.
Browser escape detection	Medium-High	Medium-High	Medium	YELLOW - add after native MVP.
Generic support for any app	Very High	Very High	High	RED - not appropriate for v1.
Infer exactly when a task is completed	Very High	Very High	High	RED - unreliable without invasive app-specific inference.
Actually remove/modify native app tabs	Not available	-	-	RED - Android provides no public API for this.
Full WebView replacement for social apps	Medium-High	High	Medium	YELLOW/RED - controllable UI but worse compatibility/login UX.

Solo-developer viability threshold

The project remains reasonable while the number of maintained detectors stays small. Supporting two high-value apps with explicit detector contracts is a product. Supporting every social app, locale, browser and A/B layout is an operations company.

5. How Native Task Tunnel Works on Android

5.1 The platform mechanism

AccessibilityService runs in the background after the user explicitly enables it and receives AccessibilityEvents when UI state changes. With window-content retrieval enabled, the service can inspect the active accessibility tree. AccessibilityNodeInfo exposes package name, text, content descriptions, class names, hierarchy and - when FLAG_REPORT_VIEW_IDS is enabled - view resource IDs. [1][2]

This is enough to build per-app surface detectors. Open-source Android blockers already use AccessibilityService to identify YouTube Shorts and Instagram Reels and navigate away. [17][18]

5.2 Detector design

Each supported app should have an isolated detector adapter that converts unstable third-party UI details into a stable internal surface enum.

Layer	Example
Raw accessibility signals	Package, view IDs, text/content descriptions, selected navigation state, hierarchy pattern
Detector output	INSTAGRAM_MESSAGES, INSTAGRAM_REELS, YOUTUBE_SHORTS, YOUTUBE_VIDEO, UNKNOWN
Policy engine	Task REPLY permits MESSAGES; task WATCH permits VIDEO/SEARCH; blocked surface -> intervention
UI response	Accessibility overlay explains mismatch and offers Return / End Tunnel / Allow Anyway

Use multiple signals rather than one magic resource ID. Resource IDs and layouts can change after third-party updates; open-source blockers explicitly report this as a known limitation, and one project removed YouTube accessibility detection after repeated breakage. [17][19]

5.3 Fail-open, not fail-hostile

- If the detector is uncertain, classify the surface as UNKNOWN and allow access rather than blocking a legitimate task.
- Never infer sensitive content or inspect message text unless absolutely necessary. Surface classification usually needs navigation structure, not message content.
- Throttle/debounce accessibility events to reduce CPU churn. Detector work should execute only for supported packages and meaningful event types.
- Keep task rules deterministic and explainable. This also reduces Play-policy risk around AccessibilityService automation. [3]

5.4 Overlay vs automatic Back

Accessibility services can create TYPE_ACCESSIBILITY_OVERLAY windows, which are designed for service-owned overlays above application content. [1][2] A deterministic automatic Back action can technically be implemented, and open-source blockers do it, but a user-driven overlay is easier to understand and safer for Play review. Google explicitly allows deterministic rule-based automation for narrow, clearly understood purposes while prohibiting autonomous planning/execution through AccessibilityService for ordinary apps. [3]

5.5 UsageStats as supporting data, not the primary trigger

UsageStatsManager can query foreground/background activity events and aggregate app usage when the user grants usage access. [6] It is useful for summaries and fallback timing. For fast interception, AccessibilityService is better: one sec states that its Android accessibility implementation is more stable, responds faster, and receives passive events instead of polling usage data. [12]

6. Recommended Technical Architecture

The first version should be deliberately boring: one Android app, no cloud backend, no LLM, no microservices. Complexity belongs in detector quality and UX.

Component	Recommended choice	Responsibility
UI	Kotlin + Jetpack Compose + Material 3	Onboarding, task gate, settings, timeline, insights
State	ViewModel + StateFlow/Coroutines	Active tunnel, permissions, detector state
Accessibility core	AccessibilityService	Foreground package events, active-window inspection, overlay trigger

Component	Recommended choice	Responsibility
Detector registry	Per-app Kotlin modules	Translate accessibility trees into stable Surface enums
Policy engine	Pure Kotlin rules	Task x surface -> allow/intercept; deterministic and testable
Event store	Room	Sessions, app switches, surfaces, interventions, overrides
Preferences	DataStore	Supported apps, tasks, session defaults, privacy settings
Usage summaries	UsageStatsManager (optional)	Cross-check duration/history; fallback signals
Backend	None for MVP	Avoid auth, privacy and operational cost
Crash/update telemetry	Opt-in later	Only after trust model and consent are clear

Core event model

A simple event schema is sufficient: timestamp, package, detected surface, active task/tunnel ID, intervention type, user choice (continue/return/end), and confidence. Do not store screen text or screenshots in the normal path.

Suggested detector interface

```
interface SurfaceDetector {
    val packageName: String
    fun detect(root: AccessibilityNodeInfo): Detection
}

data class Detection(
    val surface: Surface,
    val confidence: Float
)
```

Architecture flow

App launch -> Accessibility event -> supported package check -> Purpose Gate (if no active tunnel) -> active SessionPolicy -> app-specific SurfaceDetector -> PolicyEngine -> allow or show overlay -> EventStore -> DriftDetector/Insights. Keeping detectors and policy rules independent is the key to making third-party UI changes maintainable.

7. Reliability and Maintenance Strategy

7.1 What will break

Failure mode	Why it happens	Mitigation
View IDs change	Instagram/YouTube redesign internal resources. [17][19]	Use multiple signals; versioned detector signatures; tests; fail open.
A/B UI layouts	Different accounts/devices see different navigation.	Support several known patterns; confidence scoring.
Localization	Text/content descriptions differ by language.	MVP English-first; prefer IDs/structure over text; add locale packs later.

Failure mode	Why it happens	Mitigation
Custom/embedded surfaces	Some SurfaceView/embedded hierarchies are not discoverable through normal node search. [2]	Detect surrounding structure; document unsupported states; avoid brittle false certainty.
Accessibility permission disabled	User, OEM settings or setup issue.	Health check, onboarding, visible protection status.
Battery/background behavior	OEM-specific process policies can disrupt intervention. [12]	Compatibility guide; autostart/battery instructions only where required.
User disables blocker	Android cannot make a normal app impossible to disable. [17]	Do not promise impossibility; optional cooldown/commitment later.
Play policy review	Accessibility use is sensitive.	Make the user-facing core purpose obvious; disclosure/consent; demo video; narrow automation.

7.2 How to keep maintenance manageable

- Support a declared compatibility matrix: app, version range, locale and known surfaces. Do not market unsupported apps as "probably works."
- Create detector unit fixtures from manually captured tree fingerprints. A detector update should be testable without launching the third-party app every time.
- Use a hidden developer "Inspector" screen that prints sanitized node structure/IDs for your own device. Never upload raw trees automatically.
- Separate detector rules from business logic so app redesigns do not touch Task Tunnel state, analytics or UI.
- Optional later: download signed, non-executable detector rule packs so simple ID updates do not require a full app release. Keep behavioral data local.

7.3 Native accessibility vs WebView container

Approach	Advantages	Disadvantages	Recommendation
Native AccessibilityService	Users keep official apps; fast interception; proven by blockers	Fragile per-app detection; sensitive permission; cannot truly remove UI	PRIMARY MVP
Controlled web container	Can hide/redirect web elements much more directly; UNDOOMED demonstrates the model [14]	Web versions may lack features; login trust/compatibility concerns; feels like a replacement browser	Optional later / fallback
DNS/VPN filtering	Robust when feeds use separable hosts; no UI tree parsing	Cannot distinguish YouTube Shorts from normal YouTube when traffic shares domains; VPN conflicts [19]	Not suitable for Task Tunnel core

8. Google Play and Privacy Constraints

Current publishing requirement

As of 31 August 2026, new Android phone apps and updates submitted to Google Play must target Android 16 / API level 36 or higher. [5]

8.1 AccessibilityService declaration

A general digital-wellbeing app is not automatically an "accessibility tool." Google Play allows broader AccessibilityService use, but apps that are not disability-focused must complete the Accessibility declaration

and provide prominent in-app disclosure and affirmative consent. Google also requires a review video demonstrating setup, consent and the core feature. [3]

Google forbids ordinary apps from using AccessibilityService to autonomously initiate, plan and execute actions or decisions. Deterministic rule-based automation for a narrow, clearly understood purpose is explicitly distinguished as acceptable in the policy guidance. [3]

8.2 Package visibility

Installed-app inventory is treated as sensitive. Avoid QUERY_ALL_PACKAGES in the MVP by supporting a fixed set of packages and declaring targeted package visibility where possible. Google Play requires broad visibility to be core functionality and justifiable when narrower alternatives are insufficient. [4]

8.3 Data minimization

- Do not capture screenshots for normal detection.
- Do not persist accessibility text or private message content.
- Store only app/surface/task/session events required for the product.
- Explain exactly what the service sees and what is retained before the permission request.
- Keep default analytics local; make any future crash/usage telemetry opt-in or carefully minimized.

9. MVP Scope and Build Sequence

MVP promise

Open distracting apps with a purpose. Stay inside that purpose. Notice when you start hunting for another rabbit hole.

9.1 What v0.1 should include

1. Permission onboarding with an unusually clear accessibility disclosure and protection-health screen.
2. Purpose Gate for exactly two supported apps: Instagram and YouTube.
3. Task Tunnel policies for a small set of high-value tasks: Instagram Messages; YouTube Search/Watch; intentional Browse.
4. Detectors for Instagram Reels/Explore and YouTube Shorts, plus only the allowed surfaces needed for the tasks above.
5. User-visible interception overlay with Return, End Tunnel and Allow Anyway.
6. Optional session window + re-intervention.
7. Drift Detection across a user-selected small distraction pool.
8. Local Attention Debugger timeline and a few rolling metrics.

9.2 What should explicitly wait

Defer	Why
Machine learning / LLM coaching	Existing simple friction is already effective; ML adds cost and policy/privacy complexity before product-market validation.
All social apps	Detector maintenance would explode.
Automatic "task completed" inference	Difficult to know whether a user actually finished a meaningful task inside another app.
Account/cloud sync	No core need; increases trust, backend and compliance burden.
Hard anti-uninstall controls	Competes with AppBlock on strictness and conflicts with the autonomy-first product thesis.

Defer	Why
Complex gamification/streaks	Easy to add later; not core differentiation.
Web wrapper	Only build if native detector brittleness becomes unacceptable for a high-value task.

9.3 Recommended build order

Milestone	Deliverable	Go/no-go check
M0 - Technical spike	Accessibility service + node inspector + overlay on one phone	Can detect foreground package and show/close overlay reliably.
M1 - YouTube proof	Detect Shorts vs normal video/search	False positives are rare enough to trust on your own daily use.
M2 - Instagram proof	Detect Reels/Explore vs messaging	Same; works across repeated navigation and app restarts.
M3 - Task Tunnel	Purpose Gate + SessionPolicy + user override	The flow feels faster than manually fighting the feed.
M4 - Drift	Cross-app episode detection	Triggers on obvious app-hopping without nagging normal switching.
M5 - Attention Debugger	Local timeline + rolling metrics	Users understand what happened without raw screen-time guilt.
M6 - Hardening	Compatibility/status, disclosure, Play review materials	Permission and policy story is clear enough for external beta.

10. Testing, Validation and Success Metrics

10.1 Technical test matrix

- Start with English UI only. Add locales only after detectors are stable.
- At minimum test one Pixel-class device and one Samsung-class device; recruit beta users for other OEMs before claiming compatibility.
- Test fresh launch, background resume, split screen, picture-in-picture, notification deep links, app updates and phone reboot.
- Maintain detector regression fixtures and manual smoke tests for every supported surface.
- Track detector confidence and UNKNOWN rate locally. A rising UNKNOWN rate after a third-party update is an early warning.

10.2 Product metrics

Metric	Why it matters	Bad interpretation to avoid
Permission activation rate	The app is useless if setup scares users away.	Blaming users for declining a powerful permission.
Detector precision / false blocks	Trust is more important than aggressive blocking.	Optimizing only for number of blocked events.
Return vs Allow Anyway	Shows whether interventions are actually changing choices.	Treating every override as failure.
Task Tunnel completion/exit	Indicates whether task-shaped access is useful.	Assuming time spent under a threshold equals success.
Drift episodes per week	Measures cross-app rabbit-hole behavior.	Equating all multi-app switching with distraction.

Metric	Why it matters	Bad interpretation to avoid
Weekly retention	Tells whether friction remains tolerable.	Using streaks as the primary retention mechanic.
User-reported usefulness	Captures whether less friction is actually better.	Optimizing raw screen time regardless of user goals.

10.3 Earliest validation test

Before polishing the dashboard, make the core detector and gate work on your own daily phone. If Task Tunnel cannot reliably distinguish Shorts/Reels from useful surfaces without frequent false positives, stop or switch to the controlled-web approach. If it works technically, a small beta should answer whether users prefer temporary task-based access over static feature blocking.

11. Monetization and Positioning

Do not sell "AI screen time." Sell a concrete outcome: keep the useful part of social apps without falling into the parts you did not open them for.

Positioning concepts

- "Keep Instagram. Lose the rabbit hole."
- "Use the app you meant to use."
- "Your phone is useful. The rabbit holes are not."
- "Debug your attention."

Short-form demo concepts

- "I opened Instagram to reply to ONE message." -> Purpose Gate -> DMs allowed -> tap Reels -> "That is not what you came here for."
- "My blocker caught me trying three different apps in 54 seconds." -> Drift Detection overlay.
- "Screen time says 2h 11m. This shows how the rabbit hole actually happened." -> Attention Debugger timeline.

Pricing direction

The category contains both subscriptions and strong user resistance to expensive recurring digital-minimalism products. [20] A reasonable product experiment is a generous free core plus inexpensive premium capabilities rather than a high-price subscription from day one.

Free candidate	Premium candidate
1-2 supported apps / basic Task Tunnel	Unlimited supported detectors as they are added
Basic Drift Detection	Advanced cross-app rules and custom distraction groups
7-day local history	Longer history and advanced Attention Debugger insights
Core interventions	Advanced personalization / custom intervention library

Exact pricing should be tested after retention and support cost are known. A lifetime option is strategically attractive for users who specifically resent recurring blocker subscriptions.

12. Risks, Kill Criteria and Final Recommendation

12.1 Major risks

Risk	Severity	Reason
Third-party UI changes break detectors	HIGH	A permanent maintenance obligation; strongest solo-dev risk.
Google Play Accessibility review	HIGH	Core architecture depends on a sensitive API and clear policy justification.
False positive blocks destroy trust	HIGH	A blocker that interrupts legitimate messaging/tutorials will be uninstalled quickly.
Competitors copy task-based framing	MEDIUM	Individual mechanisms are easy to reproduce; execution and brand matter.
Users bypass/disable protection	MEDIUM	Impossible to eliminate completely without device-owner style control; product should not promise it.
Feature scope creep	HIGH	Adding every app/browser/locale can make maintenance exceed solo capacity.
Battery/performance regression	MEDIUM	Accessibility tree scans can be wasteful if unthrottled.

12.2 Kill criteria

- Kill or redesign native Task Tunnel if current YouTube/Instagram builds cannot be detected with acceptable false-positive rates using accessibility metadata available to a normal Play-distributed app.
- Kill the "compartmentalization" promise if app updates require emergency detector fixes too frequently for one maintainer.
- Kill or simplify Drift Detection if beta users experience it mainly as nagging during legitimate app switching.
- Do not ship if Play review would require deceptive accessibility-tool claims or automation outside a narrow user-defined rule set.

12.3 Final recommendation

GO - but with a constrained definition of the product

Build an Android-only proof around Purpose Gate + native Task Tunnel for Instagram/YouTube + Drift Detection. Keep all data local, use deterministic rules, and make reliability visible. Do not build ML, generic app support, cloud accounts or hard anti-uninstall behavior until the core interaction survives real daily use.

The most defensible product advantage is not a single novel feature. It is the coherent workflow: the user declares what they came to do, the phone protects that task from nearby rabbit holes, cross-app escape behavior is recognized, and the user later sees a comprehensible attention timeline instead of only a screen-time total. That combination is technically achievable by a solo Android developer if supported surfaces remain intentionally narrow.

Solo-dev checklist

- Native Android only for v1.
- Target Android 16 / API 36 for current Play submissions. [5]
- Kotlin + Compose + Room + DataStore; no backend.
- AccessibilityService is primary runtime signal; UsageStats is optional supporting data. [1][6][12]

- Exactly two supported apps at launch.
- Fail open on uncertain surface detection.
- No persisted accessibility text/screenshots.
- User-visible override on every intervention.
- Compatibility/status screen is part of MVP, not support-page cleanup.
- Product success measured by trust and intentional use, not raw screen-time reduction alone.

References

- [1] Android Developers - AccessibilityService API
<https://developer.android.com/reference/android/accessibilityservice/AccessibilityService>
- [2] Android Developers - AccessibilityNodeInfo API
<https://developer.android.com/reference/android/view/accessibility/AccessibilityNodeInfo>
- [3] Google Play - Use of the AccessibilityService API
<https://support.google.com/googleplay/android-developer/answer/10964491?hl=en>
- [4] Google Play - QUERY_ALL_PACKAGES / broad package visibility policy
<https://support.google.com/googleplay/android-developer/answer/10158779?hl=en>
- [5] Google Play - Target API level requirements
<https://support.google.com/googleplay/android-developer/answer/11926878?hl=en>
- [6] Android Developers - UsageStatsManager API
<https://developer.android.com/reference/android/app/usage/UsageStatsManager>
- [7] Orzikulova et al. - FinerMe: feature-level vs app-level social-media intervention
<https://nmsl.kaist.ac.kr/projects/finerme/>
- [8] Gruening et al. - Directing smartphone use through the self-nudge app one sec (PNAS, 2023)
<https://pmc.ncbi.nlm.nih.gov/articles/PMC9974409/>
- [9] Danish Competition and Consumer Authority - Disrupting Social Media Habits field experiment (2025)
<https://en.kfst.dk/analyses/2025/20250619-disrupting-social-media-habits-a-field-experiment-with-young-danish-consumers>
- [10] Orzikulova et al. - Time2Stop adaptive smartphone-overuse intervention
<https://arxiv.org/abs/2403.05584>
- [11] ScreenZen - Google Play listing and current feature description
<https://play.google.com/store/apps/details?hl=en&id=com.screenzen>
- [12] one sec - Android/accessibility, intervention customization, compatibility and intention documentation
<https://tutorials.one-sec.app/en/categories/683138>
- [13] one sec - Immediate vs Delayed vs Always Ask intervention modes
<https://tutorials.one-sec.app/en/articles/3974530>
- [14] UNDOOMED - feed/feature filtering and Messages Only mode
<https://undoomed.app/>
- [15] AppBlock - Android Strict Mode
<https://appblock.app/help/android/strict-mode/>
- [16] Detoxly - Google Play listing, Adaptive Friction positioning
<https://play.google.com/store/apps/details?hl=en-US&id=ai.detoxly>
- [17] atick-faisal/Shorts-Blocker - open-source AccessibilityService detector architecture
<https://github.com/atick-faisal/Shorts-Blocker>
- [18] CameronDetig/shorts-blocker - native detector approach and view-ID maintenance limitations
<https://github.com/CameronDetig/shorts-blocker>
- [19] undisputedP/ShortsBlocker - DNS limitations and removed brittle YouTube accessibility detection
<https://github.com/undisputedP/ShortsBlocker>

[20] Reddit community evidence - blockers becoming bypassed/ignored/uninstalled

https://www.reddit.com/r/ProductivityApps/comments/1w6tp2t/what_are_app_blockers_actually_supposed_to_do/

[21] Reddit community evidence - deleting Instagram and migrating attention to YouTube

https://www.reddit.com/r/digitalminimalism/comments/1rakgrs/issues_with_getting_off_instagram_and_youtube/

[22] Reddit community evidence - resistance to expensive digital-minimalism products

https://www.reddit.com/r/dumbphones/comments/1ucl9au/digital_minimalism_should_not_be_expensive/

Note on evidence: Official Android/Google Play documentation governs technical and policy claims. Academic field studies support behavioral-design claims. Commercial product documentation establishes competitor capabilities. Reddit items are used only as qualitative demand/frustration signals and are not treated as representative population evidence.